

RESEARCH ARTICLE

Optimal Controller Design for a Mobile Robot Using Genetic Algorithm and Adaptive PID Controller

JIN-HYUN PARK^{ID}

School of Mechatronics Engineering, Gyeongsang National University, Jinju 52725, Republic of Korea

e-mail: uabut@gnu.ac.kr

ABSTRACT This paper addresses the challenge of enabling a mobile robot to reach target coordinates and posture in minimal time by tackling the complexities of kinematic optimal control problems. Determining optimal control gains is analytically challenging, especially when incorporating dynamic models. The mobile robot's control system comprises two controllers: a kinematic controller, optimized using genetic algorithms and cubic polynomial interpolation, to generate the optimal trajectory, and a dynamic controller using an adaptive PID approach to ensure stable and robust performance despite variations in system parameters like the robot's weight. The proposed method demonstrates superior performance over existing methods, particularly under dynamic changes in robot characteristics. Simulation results show that the kinematic controller generates optimal trajectories more effectively than traditional techniques, while the adaptive PID controller maintains optimal performance despite disturbances and mass variations. In experiments with various disturbances, the proposed method consistently reached target coordinates with minimal timing discrepancies, outperforming comparative methods. Additionally, it significantly reduces spatial constraints and learning time. These findings contribute to the practical design of mobile robot controllers, ensuring efficient and stable operation, especially in scenarios requiring rapid derivation of optimal gains. Future work should focus on integrating kinematic and dynamic controllers into intelligent systems for mobile robots with arbitrary positions and postures, further enhancing autonomy and adaptability.

INDEX TERMS Mobile robot, optimal control, kinematic control, dynamic control, genetic algorithm, cubic polynomial interpolation.

I. INTRODUCTION

Mobile robot technology is at the forefront of innovation in modern industries and is recognized as an essential element for maximizing automation and efficiency in various fields. Particularly in applications such as logistics, manufacturing, and service robots, precise trajectory control of mobile robots is considered a key factor for improving operational efficiency and productivity. However, despite advancements in this technology, designing control systems for mobile robots remains a significant challenge due to complex dynamic characteristics and various environmental

factors [1], [2], [3]. The control system of a mobile robot is a crucial component that determines the mobile robot's performance, directly affecting its operational range, mobility, and accuracy. Research in mobile robot control can be broadly divided into two main areas: the first is generating the optimal trajectory for mobile robots, and the second is optimal control to minimize path-tracking errors while following the generated trajectory [4], [5]. Traditional control methods have primarily relied on kinematic control, focusing on adjusting the wheel speeds of the mobile robot to move it to a specific position, rotation, or along a predefined path. This approach concentrates on controlling position and orientation without considering dynamic elements such as acceleration and torque, making it suitable for low-speed operations

The associate editor coordinating the review of this manuscript and approving it for publication was Ganesh Naik^{ID}.

where inertia and dynamic effects can be ignored [6], [7], [8]. However, these approaches have limitations regarding achieving optimal control to reach the target point in minimal time. Moreover, kinematic controllers generate control output without considering the mobile robot's maximum speed and angular velocity, potentially leading to unrealistic commands that exceed these limits.

Accurately moving a mobile robot to a desired location is a complex nonlinear optimal control problem, requiring a detailed approach that integrates dynamics beyond traditional kinematic methods [9]. Existing studies have explored various techniques to enhance control performance by leveraging the dynamic models of robots. Fierro and Lewis [10] improved real-time control performance through a neural network-based controller. Meanwhile, Nandy et al. [11] implemented a modular approach to enhance accuracy and efficiency in real-time gain scheduling. Martins and Brandão [12] proposed a dynamic compensation control system applying maximum speed limits to target safety and path-tracking error reduction. However, these studies have predominantly focused on improving path-tracking performance and have not sufficiently addressed the problem of optimal trajectory generation. Regarding optimal trajectory control of mobile robots, Choi and Park [9] optimized linear controller gains using genetic algorithms and neural networks to enable mobile robots to reach the target pose quickly. However, these methods do not consider the robot's dynamic elements, maximum speed, and angular velocity, leading to potential stability issues in the control system. Recently, Park et al. [4] addressed stability issues by setting kinematic controller gains while considering dynamic constraints such as maximum speed and angular velocity. Genetic algorithms, which mimic biological evolutionary processes, provide robust solutions for complex nonlinear optimization problems, deriving optimal control gains for various trajectories of mobile robots. However, there are limitations in that they require significant time and computational resources to perform optimizations for all trajectories. Optimizing dynamic controller gains using genetic algorithms in real-world applications demands spatial constraints and time. Furthermore, when the dynamic characteristics of the system change under various environments, well-designed optimal gains can significantly weaken the system's flexibility and responsiveness. Recently, researchers have conducted many studies utilizing intelligent control techniques [13], [14], [15], [16], [17], focusing on real-time obstacle avoidance and path-tracking strategies using machine learning and vision-based approaches and addressing path-following problems with Deep Reinforcement Learning (DRL). However, these techniques still encounter challenges due to complex models, learning time, hardware, and other environmental factors.

This study aims to achieve optimal performance using PID controllers, most commonly applied in industrial and robotic control, instead of intelligent control methods. Specifically, it seeks to optimize kinematic and dynamic controllers and

proposes a method to minimize spatial constraints and the time required for controller optimization. To this end, the kinematic controller for the trajectory generation of a mobile robot is optimized by applying genetic algorithms, allowing for relatively easy and efficient trajectory planning. This study addresses these issues by optimizing the kinematic controller for trajectory generation using genetic algorithms, allowing for relatively easy and efficient trajectory planning. This approach significantly reduces the time and effort required for complex kinematic trajectory optimization and enables practical and rapid trajectory control system design. Additionally, the dynamic controller, implemented as an adaptive PID controller [18], [19], [20], enhances the ability to respond to real-time changes in environmental factors. The adaptive controller is designed to automatically adjust control parameters to maintain optimal control performance even when the dynamic characteristics of the system change. This method ensures mobile robots' stable and efficient operation in diverse environments, significantly improving the system's flexibility and responsiveness. By designing and rigorously verifying the performance of the proposed control system, this study aims to promote the development of mobile robot technology and expand its practical application scope, thereby maximizing the potential economic and social benefits of mobile robots' widespread use across various industrial fields. Such research simultaneously pursues efficiency and practicality in the design of mobile robot control systems and will significantly contribute to developing future autonomous systems.

This paper is structured as follows. Section II describes recent research and research directions. Section III introduces the nonholonomic mobile robot system and the closed-loop stability equation considering dynamic constraints. Section IV discusses the design of the mobile robot controller using genetic algorithms and cubic polynomial interpolation for optimal kinematic controller design and the design of an adaptive PID controller as a dynamic controller to match well-designed trajectory plans. Section V presents simulation results for the performance evaluation of the proposed controller. Finally, Section VI provides the conclusion.

II. RELATED WORKS

Enabling a mobile robot to reach a target position and orientation in minimal time involves addressing a kinematic optimal control problem related to the robot's initial posture. The optimal control gains for a mobile robot are challenging to derive analytically. When considering dynamic models, Kinematic analysis presents an even more complex problem. Therefore, the control problem of mobile robots has evolved into designing kinematic controllers to generate optimal trajectories and dynamic controllers to minimize path-tracking errors while following the generated trajectories.

In the design of kinematic controllers, extensive research has been conducted on trajectory generation for mobile robots using optimization techniques that explore various optimal solutions based on the robot's initial state or

constraints or through nonlinear approximation using artificial neural networks. Plaskonka [6] enhanced path accuracy and adaptability in dynamic environments by employing the Frenet-Serret formula to adjust the robot's path precisely. However, these methods are based on kinematic control and do not consider the dynamic characteristics of mobile robots, limiting their immediate and flexible application to changes in the system's state. Such kinematic controllers are suitable for mobile robots operating at low speeds. The controller's performance can vary in real systems with dynamic constraints such as the speed and acceleration of the mobile robot. Wang et al. [7] improved the mobility efficiency of mobile robots through dynamic controller design and developed dynamic equations that fit the robot's dynamics to enhance efficiency and performance. However, the dynamic model proposed in the paper is limited to specific types of mobile robots, lacking generalization for all kinds of robots. Additionally, while dynamic controllers provide precision and adaptability in control, nonlinear parameter modeling and complex design can pose challenges for real-time processing in mobile robots requiring fast trajectory control [4], [9], [21], [22].

Therefore, both kinematic controller design, considering the dynamics of mobile robots, and dynamic controller design must be considered. Al-Kamil and Szabolcsi [5] proposed an adaptive controller for the dynamic controller design of mobile robots and analyzed the performance of path planning and various path planning algorithms using motion capture technology. This paper proposed a method to optimize path planning for mobile robots using motion capture technology, but it faced issues with applicability in complex environments and limitations of the technology. Falcone [21] planned the optimal trajectory of mobile robots using Optimal Control and proposed a dynamic control method based on Model Predictive Control (MPC). However, MPC requires periodically setting paths based on the current state of the mobile robot and continuously calculating the optimal path reflecting the robot's position and speed. Recently, Park et al. [4] improved stability issues in controller design by considering dynamic constraints such as maximum speed and angular velocity of mobile robots, optimizing the optimal gains of kinematic and dynamic controllers using genetic algorithms, and enhancing performance. However, the optimized gains are selected to be optimal for specific mobile robot systems, making it difficult to appropriately respond to dynamic changes such as weight variations and disturbances in real-world application environments.

Recently, many studies have been conducted utilizing intelligent control techniques [13], [14], [15], [16], [17]. Singh et al. [15] proposed real-time obstacle avoidance and path-tracking strategies for mobile robots using machine learning and vision-based approaches. This study requires high computational resources and complex system integration due to real-time image data processing and path planning. Chai et al. [16] proposed a method using Deep Reinforcement Learning that enables mobile robots to plan and control

paths in unknown environments quickly and demonstrated the possibility of real-time applications even in complex environments. Similarly, Cao et al. [14] presented a method to solve the path-following problem for autonomous mobile robots using DRL, offering more flexible and generalized solutions than traditional methods. This method contributes to increasing autonomy and minimizing human intervention. However, both studies still face challenges related to complex models, learning time, and unknown risk factors.

Despite these challenges, this study aims to achieve optimal performance by applying PID controllers, most commonly used in industrial and robotic control. For this, designing kinematic controllers that consider the dynamics of mobile robots, along with the design of optimal dynamic controllers, is necessary. This study aims to design kinematic controller gains for optimal trajectory generation to allow mobile robots to reach the target posture accurately and quickly and to develop an adaptive PID controller that exhibits stable and robust performance against dynamic changes such as weight variations of the mobile robot.

III. KINEMATICS AND DYNAMICS OF MOBILE ROBOTS

This chapter describes the control system of a nonholonomic mobile robot. The control of a mobile robot requires optimal trajectory generation using kinematic control techniques and optimal control to minimize the path-tracking errors of the generated trajectory through dynamic control. We explain the kinematics and dynamics of mobile robots and provide an overview of the control system for mobile robots.

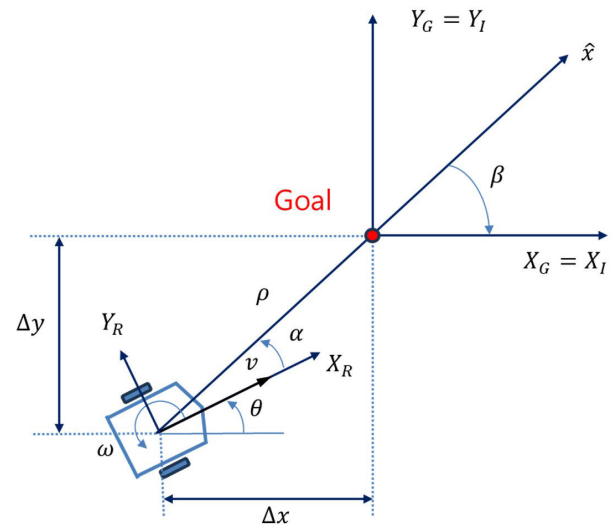


FIGURE 1. Robot kinematics and its frames of interest [22].

Fig. 1 shows the posture $([x, y, \theta]^T)$ of a mobile robot in the reference coordinate system $(X_I - Y_I)$. Here, (x, y) represents the center coordinates of the mobile robot, and θ is the angle between the robot's direction of motion and the horizontal axis. The target coordinates are $[0, 0^0, 0^0]^T$. For convenience, the target coordinate system $(X_G - Y_G)$

coincides with the reference coordinate system. θ is the angle formed by the robot's direction of motion and the horizontal axis, and $\rho = \sqrt{(\Delta x)^2 + (\Delta y)^2}$ represents the straight-line distance between the robot's current center coordinates and the destination coordinates. $\alpha = -\theta + \text{atan2}(\Delta y, \Delta x)$ is the angle between the line represented by ρ and the robot's direction of motion, and $\beta = -\theta - \alpha$ is the angle between ρ and the horizontal axis X . Equation (1) below represents the motion equation of the mobile robot [4], [9], [22].

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (1)$$

where v is the linear speed of the mobile robot, and ω is the rotational angular speed.

If conditions $\alpha \in (-\frac{\pi}{2}, \frac{\pi}{2})$ and $\beta \in (-\pi, \pi)$ are satisfied in Fig. 1, the mobile robot moves only in the forward direction. The control signal $[v, \omega]^T$ is chosen as a linear controller, as shown in Equation (2), to drive the robot from its actual set position $[\rho, \alpha, \beta]^T$ to the target position [4], [9], [22].

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} K_\rho \rho \\ K_\alpha \alpha + K_\beta \beta \end{bmatrix} \quad (2)$$

where $[K_\rho, K_\alpha, K_\beta]^T$ represent the gains of the kinematic controller.

When transforming the kinematic equation of the mobile robot with respect to $[\rho, \alpha, \beta]^T$, the closed-loop system of the mobile robot is represented by equation (3). The system has not singular point at $\rho = 0$ and a unique equilibrium point at $[\rho, \alpha, \beta]^T = [0, 0^\circ, 0^\circ]^T$. Therefore, the robot is driven to this point, the target location [4], [9], [22].

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -K_\rho \rho \cos \alpha \\ K_\rho \sin \alpha - K_\alpha \alpha - K_\beta \beta \\ -K_\rho \sin \alpha \end{bmatrix} \quad (3)$$

The velocity and angular velocity of the mobile robot are expressed in terms of the angular velocities of both wheels of the mobile robot, as shown in Equation (4).

$$\omega_r = \frac{1}{r}(v + R\omega), \quad \omega_l = \frac{1}{r}(v - R\omega) \quad (4)$$

ω_r and ω_l represent the angular velocities of the right and left wheels of the mobile robot, respectively. R is the distance from the wheels to the center of the mobile robot, and r is the radius of the wheels.

For precise control of the mobile robot, its dynamic model must be considered. In this study, the dynamic model of a nonholonomic mobile robot was constructed as shown in Equation (5) [10].

$$\begin{aligned} M(q)\ddot{q} + V_m(q, \dot{q})\dot{q} + F(\dot{q}) + G(q) + \tau_d \\ = B(q)\tau - A^T(q)\lambda \end{aligned} \quad (5)$$

where, q is $[x, y, \theta]^T$ and $M(q)$ is the positive definite symmetric inertia matrix, $V_m(q, \dot{q})$ is the centripetal and Coriolis matrix, $F(\dot{q})$ represents surface friction, $G(q)$ is the gravity vector, and τ_d is bounded by unknown disturbances. $B(q)$ represents the input transformation matrix, τ represents the input torque, $A(q)$ represents the constraint matrix, and λ is the constraint force vector. The friction force ($F(\dot{q}) = -(D + C)\dot{q}$) includes viscous friction ($-D\dot{q}$) and ground friction ($-C\dot{q}$) and is modeled so that the mobile robot can reflect its movements in a real environment [24], [25], [26]. Since the movement of the mobile robot is restricted to the horizontal plane, gravity is set to $G(q) = 0$.

The physical movement of the mobile robot determines the maximum velocity and maximum angular velocity of the controlled mobile robot. Therefore, the control commands for the mobile robot must adhere to physical constraints. Consequently, the stability condition of the closed-loop system for the linear controller can be defined as shown in Equation (6). This equation (5) represents the closed-loop stability condition from Siegwart et al. [22] and the closed-loop stability condition considering the dynamics of the mobile robot as proposed by Park et al. [4]. Therefore, the stability of the mobile robot system is assured when we select the kinematic control gains by the stability condition specified in Equation (5).

$$\begin{aligned} 0 < K_\rho &\leq \frac{v_{Max}}{\rho_{Max}} \\ K_\beta &< 0 \\ K_\alpha + \frac{5}{3}K_\beta - \frac{2}{\pi}K_\rho &> 0 \\ \frac{\pi}{2}K_\alpha - \pi K_\beta &\leq \omega_{Max} \end{aligned} \quad (6)$$

ρ_{Max} is the maximum straight-line distance between the center coordinate of the robot and the destination coordinate, v_{Max} defines the maximum velocity of the mobile robot, and ω_{Max} represents the maximum angular velocity of the mobile robot.

For controlling the mobile robot, selecting kinematic linear control gains within the range of the stability condition in Equation (6) allows the robot to reach the target point. However, these selected gains are not necessarily the optimal control gains for the mobile robot. This problem is a complex nonlinear control problem that is difficult to solve using general optimal control theory [4], [9], [22]. The first row of Equation (3) is a linear time-varying differential equation concerning ρ , where $\alpha(t)$ is a time-varying parameter. Linear differential equations exhibit a consistent pattern of solutions only concerning the initial value $\rho(0)$, regardless of whether they involve time-varying or time-invariant elements. The second and third rows of Equation (3) are nonlinear differential equations concerning $\rho(t)$ and $\beta(t)$, which can yield entirely different results depending on the magnitudes of the initial values $\alpha(0)$ and $\beta(0)$ [9]. Therefore, optimal control gains must be derived to reach the target point quickly and accurately.

In the field of mobile robot control, research has combined studies on generating optimal trajectories using kinematic control techniques with dynamic control techniques that minimize path-tracking errors as the mobile robot follows the generated trajectory. Recently, the study by Park et al. [4] laid the foundation for addressing issues inherent in control techniques composed of individual controllers by integrating these two methods. They derived a closed-loop stability condition for gains that ensure system stability by considering the dynamics of the mobile robot, addressing command values for kinematic speed and angular velocity that exceed physical limits. Genetic algorithms were used to determine the optimal gains within this derived stability condition, and the dynamic controller's gains were also selected using genetic algorithms. This approach provides optimal gains for mobile robots with various postures. It ensures stable and predictable operation, enhances adaptability to environmental changes, and extends the robot's lifespan. However, a dynamic controller with fixed optimized gains may struggle to adapt accurately to variations in system parameters, such as changes in the robot's mass.

Additionally, selecting gains for a dynamic controller using genetic algorithms requires substantial time and effort. This process demands significant space and time in real systems, unlike in simulations. Therefore, this study applies the gradient descent method for dynamic control, allowing for the selection of optimal gains for kinematic control without time and space constraints. This method adaptively selects controller gains in response to variations in system parameters.

IV. DESIGN OF MOBILE ROBOT CONTROLLERS

This section will discuss the design of controllers for mobile robots. The design of a mobile robot controller involves broadly dividing the process into two approaches. The first approach entails optimizing the kinematic controller gains offline to enhance the trajectory of the mobile robot and then using these optimized gains to generate the robot's optimal trajectory. The second approach involves reducing the tracking error of the mobile robot by employing an online adaptive PID controller based on the optimized trajectory. The first subsection will explain the design of controllers using genetic algorithms and cubic interpolation for optimal kinematic controller design. The second subsection will cover the design of an adaptive PID controller as a dynamic controller aligned with a well-designed trajectory plan.

A. DESIGN OF KINEMATIC CONTROLLERS FOR MOBILE ROBOTS USING GENETIC ALGORITHMS

The design of kinematic controllers is akin to designing the optimal trajectory for mobile robots. When the kinematic controller gains of a mobile robot are selected, as shown in Equation (6), the robot can reach the target point without exceeding its maximum speed and angular velocity. However, these gains are not necessarily optimal. This issue involves

highly complex nonlinear control, which is challenging to solve using general optimal control theory [4], [23]. This study aims to determine the kinematic control gains that allow the mobile robot to reach the target point most quickly and accurately using the genetic algorithm, a widely recognized global search algorithm [6].

The genetic algorithm [27], [28] was founded on the basic concepts and mathematical foundations established by Holland [28] in the 1960s, based on natural selection and genetics. Genetic algorithms mimic biological evolutionary processes and have been successfully applied in many fields, such as optimization problems, system identification, and image processing, proving their effectiveness. Notably, genetic algorithms perform optimization by designing a fitness function, which allows them to avoid constraints such as the differentiability of the objective function required by traditional optimization methods. Moreover, no additional information other than the fitness function is needed, representing a significant advantage in applying optimization problems. Unlike conventional optimization methods, genetic algorithms create a population of solutions within a given search space and perform parallel processing of multiple search points, increasing the likelihood of finding a global solution. Since genetic algorithms search for optimal solutions through probabilistic rather than deterministic methods, they are less likely to become trapped in local minima [4], [28]. Genetic algorithms consist of reproduction, crossover, and mutation based on an initial population. They evaluate an individual's fitness and probabilistically select and replicate individuals with high fitness to evolve to the next generation [4], [28]. The mutation operator maintains the diversity of the solution population, and the crossover rate is set to ensure rapid convergence to the optimal solution. A high crossover rate promotes gene exchange between parent individuals to produce offspring with better solutions, helping the solution population quickly converge to the optimal solution [4], [27], [28].

The design of kinematic controllers is independent of the dynamics of the mobile robot. Therefore, Fig. 2 implements the kinematic control system using only the kinematic equations of the mobile robot. Of course, if the conditions of the closed-loop stability equation (6) are satisfied, stable control performance will be achieved. However, since Equation (3) is a nonlinear differential equation, the convergence trajectory $([\rho(t), \alpha(t), \beta(t)]^T)$ can take very different forms depending on the initial conditions $([\rho(0), \alpha(0), \beta(0)]^T)$, even with the same control gains $([K_\rho, K_\alpha, K_\beta]^T)$. Consequently, the convergence speed also varies. Thus, the minimum-time control gain for the given initial conditions $([\rho(0), \alpha(0), \beta(0)]^T)$ is determined uniquely based on these initial conditions, illustrating the necessity of adapting control gains to specific scenarios.

Therefore, $[K_\rho, K_\alpha, K_\beta]^T$ is considered as the chromosome, or search variable, in the genetic algorithm. By numerically solving the nonlinear differential Equation (3) based on these search variable values, a fitness function is designed

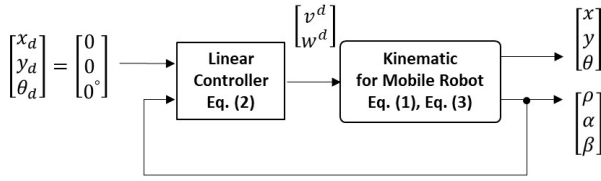


FIGURE 2. Kinematic control for mobile robot.

using the time (T_f) it takes to converge to the target point $[\rho, \alpha, \beta]^T = [0, 0^\circ, 0^\circ]^T$ and the performance error to determine the optimal gains. Equation (7) represents the fitness function of the genetic algorithm. The fitness function is set to ensure that the mobile robot achieves the fastest and most accurate posture at the target point.

$$F = at_f + b(|\rho(t_f)| + |\alpha(t_f)| + |\beta(t_f)|) \quad (7)$$

where T_f is the final target arrival time, and $\alpha(t_f)$ and $\beta(t_f)$ are the posture angles of the mobile robot at T_f . a and b represent weights.

Although $\alpha(0)$ and $\beta(0)$ can be subdivided into infinitely many segments within the intervals $(-\pi/2, \pi/2]$ and $(-\pi, \pi]$, respectively, finding the optimal controller gains $[K_\rho, K_\alpha, K_\beta]^T$ would be computationally infeasible with infinite subdivisions. Therefore, $\alpha(0)$ and $\beta(0)$ intervals are discretized appropriately to generate optimal gains. Thus, as shown in Equation (8), $\alpha(0)$ and $\beta(0)$ are discretized into N_α and N_β points, respectively, to create grid points [4].

$$\begin{aligned} \alpha(0) &\in \left\{ -\frac{\pi}{2} + \frac{\pi}{N_\alpha}, -\frac{\pi}{2} + \frac{2\pi}{N_\alpha}, \dots, -\frac{\pi}{2} + \frac{N_\alpha\pi}{N_\alpha} \right\}, \\ \beta(0) &\in \left\{ -\pi + \frac{2\pi}{N_\beta}, -\pi + \frac{2(2\pi)}{N_\beta}, \dots, -\pi + \frac{N_\beta(2\pi)}{N_\beta} \right\} \end{aligned} \quad (8)$$

The process can be expressed in algorithmic form as follows.

Step 1) Initialize: $\rho(0) = 5, \alpha(0) = -\frac{\pi}{2} + \frac{\pi}{N_\alpha}$

Step 1-1) $\beta(0) = -\pi + \frac{2\pi}{N_\beta}$

Step 2) In the genetic algorithm, set the kinematic control gains $[K_\rho, K_\alpha, K_\beta]^T$ as a search parameter and ensure that the stability condition (6) is satisfied. Solve the nonlinear differential equation (3) numerically from the initial value $[\rho(0), \alpha(0), \beta(0)]^T$, and calculate the fitness function of equation (8).

Step 3) Run the genetic algorithm until the fitness function is minimized to find the optimal gains $[K_\rho, K_\alpha, K_\beta]^T$ at the initial value $[\rho(0), \alpha(0), \beta(0)]^T$.

Step 4) If $\beta(0) = \pi$, proceed to Step 5). Otherwise, set $\beta(0) = \beta(0) + \frac{2\pi}{N_\beta}$ and return to Step 2).

Step 5) If $\alpha(0) = \frac{\pi}{2}$, terminate the process. Otherwise, set $\alpha(0) = \alpha(0) + \frac{\pi}{N_\alpha}$ and return to Step 1-1).

The above algorithm uses the genetic algorithm for each grid point to find the control gains $[K_\rho, K_\alpha, K_\beta]^T$ that minimize the fitness value while satisfying the stability

condition of the closed-loop system. For initial postures of the mobile robot that are not discretized, an interpolation algorithm can generate a gain value. Fig. 3 schematically illustrates the process of generating optimal gains for discretized initial postures of the mobile robot using the genetic algorithm and generating gain values for arbitrary initial postures that are not discretized using a cubic interpolation algorithm.

One of the representative algorithms for cubic polynomial interpolation is Cubic Spline Interpolation. Cubic Spline Interpolation models each segment with a cubic polynomial to smoothly connect the entire set of data points. In the cubic polynomial interpolation algorithm, the initial posture angle data $([\alpha(0) \beta(0)]^T)$ of the mobile robot are selected as the x-y axis sample grid points, and the kinematic linear control gains $[K_\rho, K_\alpha, K_\beta]^T$ are used as the sample values to perform two-dimensional cubic interpolation. The basis functions used in cubic interpolation are selected as cubic polynomials for each axis direction sample data, as described in Equation (9). The nodes for interpolation are selected as $(N_\alpha - 1)$ and $(N_\beta - 1)$ based on the grid intervals of $\alpha(0)$ and $\beta(0)$.

$$\begin{aligned} \text{Each segment } [\alpha_i, \alpha_{i+1}] : S_i(\alpha) \\ = a_i(\alpha - \alpha_i)^3 + b_i(\alpha - \alpha_i)^2 + c_i(\alpha - \alpha_i) + d_i \\ \text{Each segment } [\beta_i, \beta_{i+1}] : S_i(\beta) \\ = e_i(\beta - \beta_i)^3 + f_i(\beta - \beta_i)^2 + g_i(\beta - \beta_i) + h_i \end{aligned} \quad (9)$$

where α_i and β_i represent the initial posture angles of the mobile robot, while $a_i, b_i, c_i, d_i, e_i, f_i, g_i$, and h_i denote the coefficients of the cubic polynomial.

The optimized control gains do not differ significantly from adjacent optimal gains, so using a cubic interpolation algorithm alone has little impact on the control performance of the mobile robot. For interpolation, the input variables $([\alpha(0) \beta(0)]^T)$ are used, and the variables $[K_\rho, K_\alpha, K_\beta]^T$ for interpolation are each represented by a cubic polynomial. Values modeled by cubic polynomials provide smoother and more accurate results than linear interpolation.

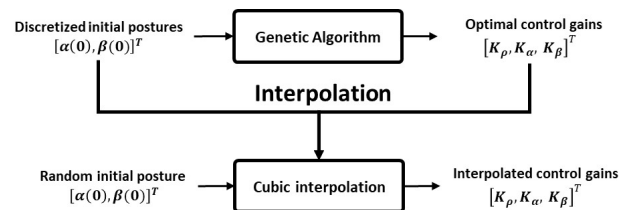


FIGURE 3. GA-based cubic polynomial interpolation.

B. DESIGN OF DYNAMIC CONTROLLERS FOR MOBILE ROBOTS USING GRADIENT DESCENT LEARNING

In the previous section, we generated the trajectory using the optimal kinematic gains designed offline and applied an

online adaptive PID controller as the dynamic controller of the mobile robot based on the optimized trajectory. Fig. 4 is a block diagram of a dynamic control system for a mobile robot featuring an adaptive PID controller based on the gradient descent method. The control system regulates the angular velocities of both wheels of the mobile robot according to the control commands from the previously designed kinematic controller to move the robot to the target coordinates. $[\omega_r^d, \omega_l^d]^T$ represents the desired angular velocity values for both wheels of the mobile robot and is designed using an adaptive PID controller based on angular velocity errors. Generally, PID control methods with fixed gains struggle to achieve appropriate gains and performance when sudden disturbances occur or when dynamic characteristics, such as the weight of the mobile robot, change, making it difficult to expect high-performance control [23]. Therefore, this study proposes an adaptive PID controller that automatically tunes the PID gains to achieve robust control performance for mobile robots. Since we cannot directly obtain the PID controller gains from the velocity error of the mobile robot, we update them to minimize the newly defined error function. We use small, constant gains to prevent issues caused by sudden changes in gains and enhance system stability.

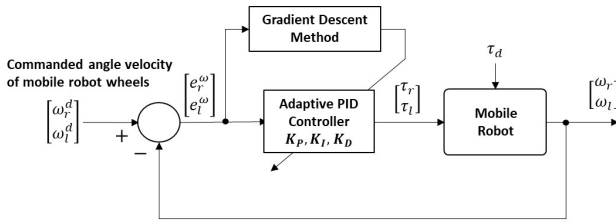


FIGURE 4. Dynamic control for a mobile robot.

The control input $([\tau_r, \tau_l]^T)$ for the mobile robot can be designed as shown in Equation (10). The initial gains in an adaptive PID controller are crucial as they can significantly affect performance. Therefore, to ensure a stable startup in the initial control phase, the controller is set with fixed gains (K_P, K_I, K_D) for the controller as shown in Equation (10), and to use the gradient descent method to learn the changes (dK_1, dK_2, dK_3) in the controller gains. Gradient descent for parameter updates ensures the boundedness of parameters and generally indicates that the parameter estimates of the adaptive mechanism do not lead to unbounded increases.

$$\tau_j(t) = (K_P + dK_1)e_j(t) + (K_I + dK_2) \int_0^t e_j(\tau) d\tau + (K_D + dK_3)\dot{e}_j(t) \quad (10)$$

where $\tau_j(t) = [\tau_r, \tau_l]^T$, and K_P, K_I , and K_D represent the constant proportional, integral, and derivative gains of the PID controller, while dK_1, dK_2 , and dK_3 represent the changes in gains to be learned.

The error function for parameter update is constructed using the error signal between the desired angular velocity of the mobile robot's wheels and the angular velocity output

by the robot's dynamics. We adjust the PID controller gains change dK_1, dK_2 , and dK_3 to minimize the error. Equation (11) shows the formulation of the error function $E_p(k)$.

$$E_p(k) = \frac{1}{2} \sum_{j=1}^2 [q_j^d(k) - q_j(k)]^2 = \frac{1}{2} \sum_{j=1}^2 e_j(k)^2 \quad (11)$$

where $q^d(k) = [q_1^d(k) q_2^d(k)]^T = [\omega_r^d(k) \omega_l^d(k)]^T$ are the target angular velocity values for the right and left wheels of the mobile robot, and $(k) = [q_1(k) q_2(k)]^T = [\omega_r(k) \omega_l(k)]^T$ are the measured angular velocity values. Therefore, $e_j(k)$ represents the angular velocity error of the mobile robot's wheels at sampling time k .

Updates $(\Delta dK_1, \Delta dK_2, \Delta dK_3)$ for changes (dK_1, dK_2, dK_3) in PID controller gains are defined using the chain rule as follows:

$$\Delta dK_i = -\zeta_i \frac{\partial E_p(k)}{\partial dK_i(k)} = -\zeta_i \frac{\partial E_p(k)}{\partial q_j(k)} \frac{\partial q_j(k)}{\partial \tau_j(k)} \frac{\partial \tau_j(k)}{\partial dK_i(k)} \quad (12)$$

where ζ_i represents the learning rate, $i = 1, 2, 3$, and $j = 1, 2$.

Each term in Equation (12) is organized as follows. $\partial E_p(k)/\partial q_j(k)$ is derived from Equation (11) as shown in Equation (13):

$$\frac{\partial E_p(k)}{\partial q_j(k)} = -(q_j^d(k) - q_j(k)) = -e_j(k) \quad (13)$$

$\partial q_j(k)/\partial \tau_j(k)$ represents the Jacobian of the mobile robot system, which is difficult to determine precisely. Especially in real control systems, it cannot be known exactly. Therefore, the mathematical approximation $\Delta q_j(k)/\Delta \tau_j(k)$ is used. Additionally, due to the nature of the gradient descent learning method, the sign of this value is more important than its magnitude, and it is used as modified in Equation (14). This method also applies to simulation experiments in situations that may occur in actual experiments.

$$\frac{\partial q_j(k)}{\partial \tau_j(k)} := \text{sgn} \left(\frac{\Delta q_j(k)}{\Delta \tau_j(k)} \right) \quad (14)$$

where $\Delta q_j(k) = q_j(k-1) - q_j(k-2)$ and $\Delta \tau_j(k) = \tau_j(k-1) - \tau_j(k-2)$.

To determine $\partial \tau_j(k)/\partial dK_i(k)$ we first rearrange Equation (10) for a discrete-time system as follows.

$$\begin{aligned} \tau_j(k) &= \tau_j(k-1) + (K_P + dK_1)(e_j(k) - e_j(k-1)) \\ &\quad + (K_I + dK_2)e_j(k) \\ &\quad + (K_D + dK_3)(e_j(k) - 2e_j(k-1) + e_j(k-2)) \end{aligned} \quad (15)$$

From Equation (15), $\partial \tau_j(k)/\partial dK_i(k)$ is derived as follows.

$$\begin{aligned} \frac{\partial \tau_j(k)}{\partial dK_1(k)} &= e_j(k) - e_j(k-1) \\ \frac{\partial \tau_j(k)}{\partial dK_2(k)} &= e_j(k) \\ \frac{\partial \tau_j(k)}{\partial dK_3(k)} &= e_j(k) - 2e_j(k-1) + e_j(k-2) \end{aligned} \quad (16)$$

Therefore, the update equation for the online automatic tuning of the changes in the adaptive PID controller gains $dK_i(k)$ is as follows using equation (12):

$$dK_i(k+1) = dK_i(k) + \Delta dK_i(k) \quad (17)$$

To evaluate the system stability of the dynamic controller through gradient descent learning, we define a Lyapunov function as shown in Equation (18). The Lyapunov function is defined as follows to reflect the trajectory error of the system.

$$\begin{aligned} V(e, \tilde{K}) &= \frac{1}{2} e^T e + \frac{1}{2} \tilde{K}_p^T \zeta_p^{-1} \tilde{K}_p + \frac{1}{2} \tilde{K}_i^T \zeta_i^{-1} \tilde{K}_i \\ &\quad + \frac{1}{2} \tilde{K}_d^T \zeta_d^{-1} \tilde{K}_d \end{aligned} \quad (18)$$

where $e = q^d - q$ is the trajectory error. $\tilde{K}_p$, $\tilde{K}_i$, and $\tilde{K}_d$ are the estimation errors of the proportional, integral, and derivative gains, respectively and ζ_p , ζ_i , and ζ_d are the learning rates for each gain, which are positive definite.

By taking the time derivative of the Lyapunov function for stability evaluation, we obtain Equation (19).

$$\dot{V} = e^T \dot{e} + \tilde{K}_p^T \zeta_p^{-1} \dot{\tilde{K}}_p + \tilde{K}_i^T \zeta_i^{-1} \dot{\tilde{K}}_i + \tilde{K}_d^T \zeta_d^{-1} \dot{\tilde{K}}_d \quad (19)$$

Since the gain adjustment of the PID controller through gradient descent learning is as shown in Equation (12), $\tilde{K}_p$, $\tilde{K}_i$, and $\tilde{K}_d$ can be used as shown in Equation (20).

$$\tilde{K}_x^T \zeta_x^{-1} \dot{\tilde{K}}_x = -\tilde{K}_x^T \frac{\partial E_p}{\partial K_x} \quad (20)$$

where $x \in \{p, i, d\}$.

Therefore, substituting Equation (20) into Equation (19) gives the Lyapunov derivative function as shown in Equation (21).

$$\dot{V} = e^T \dot{e} - \tilde{K}_p^T \frac{\partial E_p}{\partial K_p} - \tilde{K}_i^T \frac{\partial E_p}{\partial K_i} - \tilde{K}_d^T \frac{\partial E_p}{\partial K_d} \quad (21)$$

In Equation (21), $(e^T \dot{e})$ represents the error dynamics term, indicating the instantaneous change in error, and is directly influenced by the gains $[K_p, K_i, K_d]^T$. These gains are designed to reduce the error over time through the adaptive PID controller. The terms $-\tilde{K}_p^T \frac{\partial E_p}{\partial K_p}$, $-\tilde{K}_i^T \frac{\partial E_p}{\partial K_i}$, and $-\tilde{K}_d^T \frac{\partial E_p}{\partial K_d}$ represent the gain adjustments through the gradient descent method, and they are designed to be negative, which means they are oriented to reduce the error, indicating that the adaptation process decreases the error. Therefore, it is expressed in a simplified form, as shown in Equation (22).

$$\dot{V} = -e^T K_p e - \left(\int e dt \right)^T K_i \int e dt - \dot{e}^T K_d \dot{e} \quad (22)$$

The gains (K_p, K_i, K_d) are positive definite, and each term represents the proportional, derivative, and integral components, respectively. These components are adjusted to be negative, ensuring that the Lyapunov derivative

$\dot{V} \leq 0$, which implies a decrease in the system's energy. The adaptive PID controller provides Lyapunov stability for the mobile robot system through gradient descent learning. This approach allows the system to adapt to various environmental changes and adjust the control gains in real time to minimize trajectory error, maintaining stability through error reduction without explicitly relying on dynamic equations.

A dynamic model of a nonholonomic mobile robot, like that described in Equation (5), is required to design a controller that considers the robot's dynamics. The target coordinates are $[x_d, y_d, \theta_d]^T = [0, 0, 0^\circ]^T$. The control system consists of the previously designed kinematic controller and an adaptive PID controller that considers the dynamics of the mobile robot to move the robot to the target posture. The overall control system configuration is depicted in Fig. 5. The kinematic controller derives the optimal gains using an offline genetic algorithm, as shown in Figure 2, and then selects the linear controller gains using a cubic interpolation algorithm based on the optimal gain. The selected linear controller gains determine the mobile robot's desired speed and angular velocity from the kinematic linear controller Equation (2), which determines the wheel's angular velocities. The determined wheel angular velocities act as dynamic controller angular velocity commands to operate the adaptive PID controller of the mobile robot. The adaptive PID controller maintains robust control performance even when dynamic characteristics, such as disturbances and the weight of the mobile robot, change.

V. EXPERIMENTAL METHOD AND SETUP

This section describes the system parameters of the mobile robot for simulation and the hardware specifications used in the experiments. The design of the kinematic controller for the mobile robot involves using a genetic algorithm to determine the optimal gains at discretized initial positions. Appropriate gains are generated using a cubic polynomial interpolation algorithm for arbitrary initial positions that are not discretized. The kinematic gains obtained from the cubic polynomial interpolation algorithm are used to generate the trajectory of the mobile robot. The dynamic controller, designed as an adaptive PID controller, minimizes tracking errors in alignment with the well-designed trajectory of the mobile robot. This setup demonstrates robust control performance even with changes in the robot's mass.

A. EXPERIMENTAL METHOD AND SETUP

The mobile robot used for the simulation experiment is equipped with two independent drive motors that offer excellent flexibility and versatility, allowing it to move freely in multiple directions. This capability enables efficient operation in complex environments, making it applicable in various fields such as logistics, healthcare, and manufacturing. In this study, we selected the following realistic-sized mobile robot parameters for control. The matrix values of the mobile

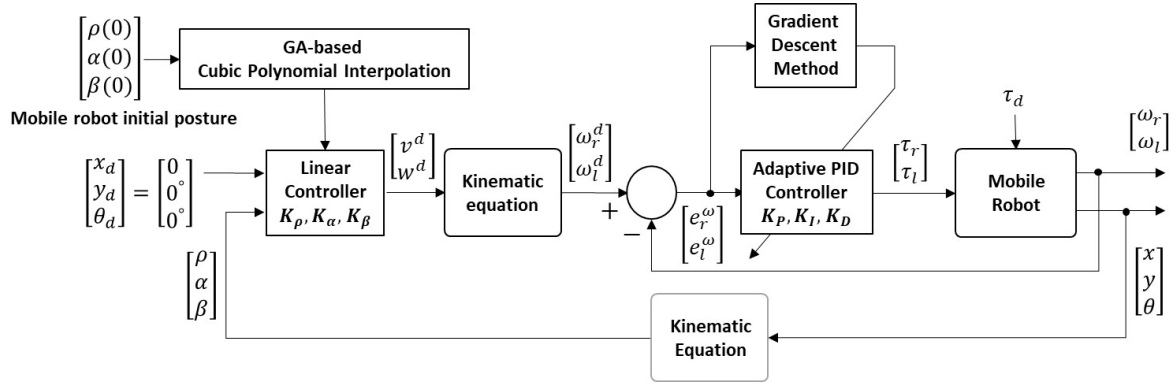


FIGURE 5. Control system for a mobile robot.

robot's dynamic model in Equation (5) are as follows [9]:

$$\begin{aligned}
 M(q) &= \begin{bmatrix} m & 0 & md \sin \theta \\ 0 & m & -md \cos \theta \\ md \sin \theta & -md \cos \theta & I \end{bmatrix} \\
 V_m(q, \dot{q}) &= \begin{bmatrix} 0 & 0 & md \dot{\theta} \cos \theta \\ 0 & 0 & md \dot{\theta} \sin \theta \\ 0 & 0 & 0 \end{bmatrix}, B(q) = \frac{1}{r} \begin{bmatrix} \cos \theta & \cos \theta \\ \sin \theta & \sin \theta \\ R & -R \end{bmatrix} \\
 F(\dot{q}) &= -(D + C) \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} \\
 D &= \begin{bmatrix} k_{d1}m + c_1 & 0 & 0 \\ 0 & k_{d1}m + c_1 & 0 \\ 0 & 0 & k_{d2}m + c_2 \end{bmatrix} \\
 C &= \begin{bmatrix} k_{c1}m + e_1 & 0 & 0 \\ 0 & k_{d1}m + e_1 & 0 \\ 0 & 0 & k_{d2}m + e_2 \end{bmatrix} \\
 A(q)^T &= \begin{bmatrix} -\sin \theta \\ \cos \theta \\ -d \end{bmatrix}, \lambda = -m(\dot{x}_c \cos \theta + \dot{y}_c \sin \theta)\dot{\theta} \quad (23)
 \end{aligned}$$

In the above matrices, the mass (m) of the mobile robot is 20 kg, the wheel radius (r) is 0.165 m, and the distance (R) from the center of the mobile robot to the wheels is 0.3 m. The center of mass is located between the wheels. The frictional force ($F(\dot{q})$) includes viscous friction ($-D\dot{q}$) and ground friction ($-C\dot{q}$), with the viscous and ground friction coefficients set to have a linear relationship with the mass. This approach models the motion of the mobile robot more accurately. The friction coefficient for simulations varies depending on the robot's design and environmental conditions. It is necessary to accurately set these values to optimize the control system of the mobile robot and ensure stable operation. Generally, mobile robots weighing between 20 kg and 100 kg are considered medium-sized robots. The friction coefficients assume that the mobile robot used in the simulation is medium-sized and that the driving environment is an indoor concrete floor [24], [25], [26]. For medium-sized robots, the linear coefficients of the viscous friction coefficient ($[k_{d1}, k_{d2}]^T$) are generally in the range

of 0.001 to 0.01 Ns/m/kg and 0.0005 to 0.005 Ns/m/kg, the constant terms ($[c_1, c_2]^T$) are in the range of 0.05 to 0.2 Ns/m and 0.02 to 0.1 Ns/m, respectively. Additionally, the linear coefficients of the ground friction ($[k_{c1}, k_{c2}]^T$) are generally in the range of 0.0001 to 0.01 N/kg and 0.00005 to 0.005 Nm/kg, the constant terms ($[e_1, e_2]^T$) are in the range of 0.05 to 0.15 N and 0.03 to 0.1 Nm, respectively. Therefore, for the simulations, the midpoint values within the friction coefficient range for medium-sized robots were selected as follows. The simulation was conducted with the viscous friction coefficients set to $[k_{d1}, k_{d2}]^T = [0.005, 0.0025]^T$ and $[c_1, c_2]^T = [0.12, 0.06]^T$, and the ground friction coefficients set to $[k_{c1}, k_{c2}]^T = [0.005, 0.005]^T$ and $[e_1, e_2]^T = [0.1, 0.07]^T$.

The output of the left and right motors used in the robot's wheels is 400 W, with a rated speed of 3000 rpm and a gear ratio of 20:1. Therefore, if the mobile robot is designed to move only in the forward direction, the motor's maximum speed (v_{Max}) and maximum angular velocity (ω_{Max}) are designed to be 2.59 m/s and 4.318 rad/s, respectively. The initial position ($\rho(0)$) for all mobile robots is set to 5 m, the most practical distance for real mobile robots in logistics and manufacturing. The sampling time for control is set to 10 ms.

The hardware specifications used for the mobile robot simulation are an Intel Xeon w5-3433 CPU, 128 GB RAM, and an NVIDIA RTX A6000. The selection of optimized gains and the simulation were performed using the genetic algorithm toolbox embedded in MathWorks' MATLAB® [29]. These hardware specifications provide a robust platform for running simulations and optimizing control gains effectively.

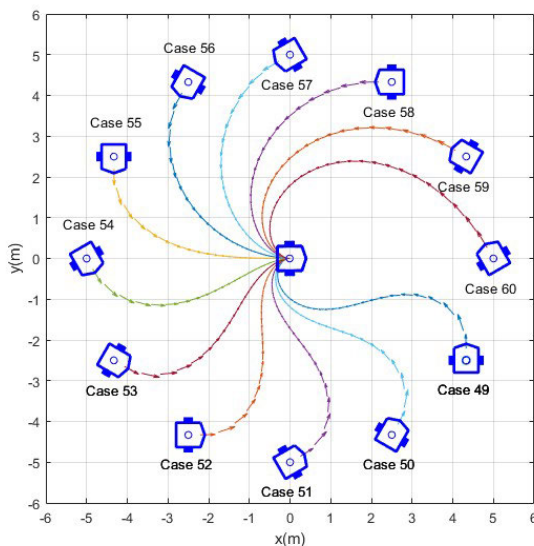
B. KINEMATIC OPTIMAL CONTROL GAIN DESIGN AND CONTROL GAIN INTERPOLATION

Moving a mobile robot from an arbitrary initial posture to a desired position requires solving an optimal control problem for a complex nonlinear system like Equation (3) [4]. As described in Equation (6), the linear controller stability conditions are fundamental for generating optimal trajectories considering dynamics. In this section, we use a

TABLE 1. Optimal control gains from the genetic algorithm.

Case	Initial Position		Optimal control gains, traveling time, and fitness				
	α_0	β_0	K_ρ	K_α	K_β	$T_f(\text{sec.})$	$F(\text{fitness})$
1	-60°	-150°	0.5178	1.1583	-0.2721	13.73	13.7590
2	-60°	-120°	0.5180	1.1079	-0.2254	13.17	13.1976
3	-60°	-90°	0.5180	0.9407	-0.0856	12.76	12.7894
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
49	60°	-150°	0.5180	1.2774	-0.3908	14.00	14.0320
50	60°	-120°	0.5174	1.3259	-0.4234	13.40	13.4262
51	60°	-90°	0.5180	1.3990	-0.4624	12.88	12.9148
52	60°	-60°	0.5178	1.5180	-0.5192	12.50	12.5436
53	60°	-30°	0.5179	1.6097	-0.5594	13.27	12.3103
54	60°	0°	0.5178	1.5842	-0.5735	12.21	12.2331
55	60°	30°	0.5180	1.8647	-0.4418	12.20	12.2238
56	60°	60°	0.5179	1.4650	-0.3818	12.45	12.4813
57	60°	90°	0.5180	1.2337	-0.2717	12.76	12.7996
58	60°	120°	0.5170	1.1205	-0.2341	13.18	13.2068
59	60°	150°	0.5169	1.2411	-0.3242	13.80	13.8317
60	60°	180°	0.5179	1.1138	-0.2571	14.37	14.4134
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
70	90°	120°	0.5178	1.1402	-0.2159	13.47	13.5136
71	90°	150°	0.5178	1.0825	-0.2109	14.02	14.0511
72	90°	180°	0.5179	1.1328	-0.2550	14.68	14.7120

genetic algorithm capable of global search to select optimal gains for generating trajectories from discretized specific postures of the mobile robot. Appropriate gains are chosen for arbitrary, unlearned postures using a computationally simple and easy-to-apply cubic polynomial approximation method.

**FIGURE 6.** The optimal path of the mobile robot when the initial conditions are $\alpha(0) = 60^\circ$ and $-180^\circ < \beta(0) \leq 180^\circ$. The initial posture of the mobile robot is shown in Table 1.

To find the optimal control gains of the mobile robot controller, the intervals of $\alpha(0)$ and $\beta(0)$ are discretized by selecting $N_\alpha = 6$ and $N_\beta = 12$. A genetic algorithm is used in the two-dimensional space grid points to obtain optimal control gains $[K_\rho, K_\alpha, K_\beta]^T$. The parameters of the genetic algorithm are set to 50 generations, a population size of 50,

a mutation rate of 0.1, and a crossover rate of 0.8. The fitness function in Equation (7) is designed to ensure that the mobile robot achieves the fastest and most accurate posture at the target point, with weights $[a, b]$ set at $[0, 0]$.

Table 1 shows the optimal gains, final arrival time, and fitness selected by the genetic algorithm at the discretized grid points. The linear controller gains vary depending on the initial posture of the mobile robot, and arrival times range from 12.02 to 14.68 s. This variation in optimal gains and trajectories is due to differences in the initial postures of the mobile robot. More complex trajectory paths result in relatively longer operation times for the mobile robot. Therefore, the optimal gains $[K_\rho, K_\alpha, K_\beta]^T$ for the linear kinematic controller should be determined differently according to the initial position of the mobile robot. Differences in fitness and arrival time arise from errors in the robot's posture at the final arrival time.

Fig. 6 shows the optimal trajectories derived using the optimized control gains obtained by the genetic algorithm, specifically for the initial conditions of the mobile robot where $\alpha(0) = 60^\circ$ and $-180^\circ < \beta(0) \leq 180^\circ$ among the 72 two-dimensional space grid points. Velocity vectors are represented as arrows along the trajectory path at 200 ms intervals. We can observe that the mobile robot starts at a high speed, and the speed decreases as it converges to the target point.

The genetic algorithm is handy for finding the optimal kinematic control gains $[K_\rho, K_\alpha, K_\beta]^T$ for specific initial postures of the mobile robot. However, calculating the optimal control gains for every initial posture is inefficient. This process requires significant computation time and effort, which limits the ability to derive optimal control gains in real time for all initial postures. Therefore, we implemented a cubic polynomial interpolation algorithm that is

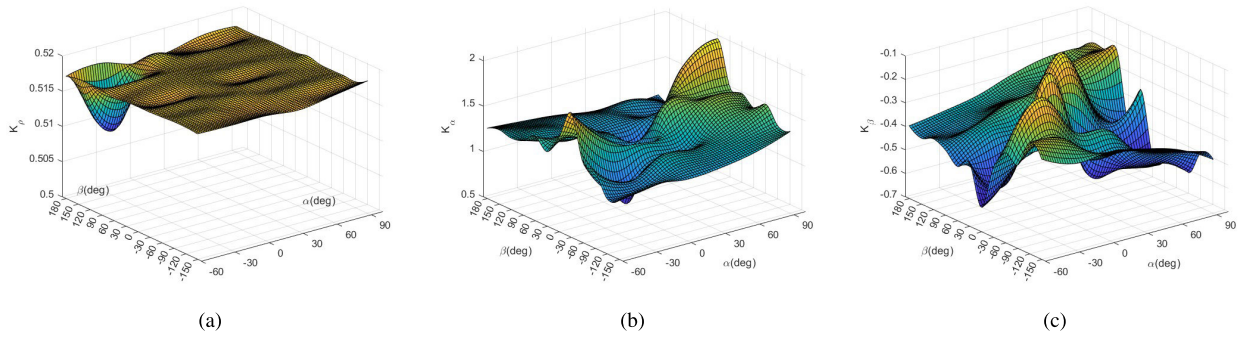


FIGURE 7. Control gains by cubic polynomial interpolation algorithm. (a) K_ρ (b) K_α (c) K_β .

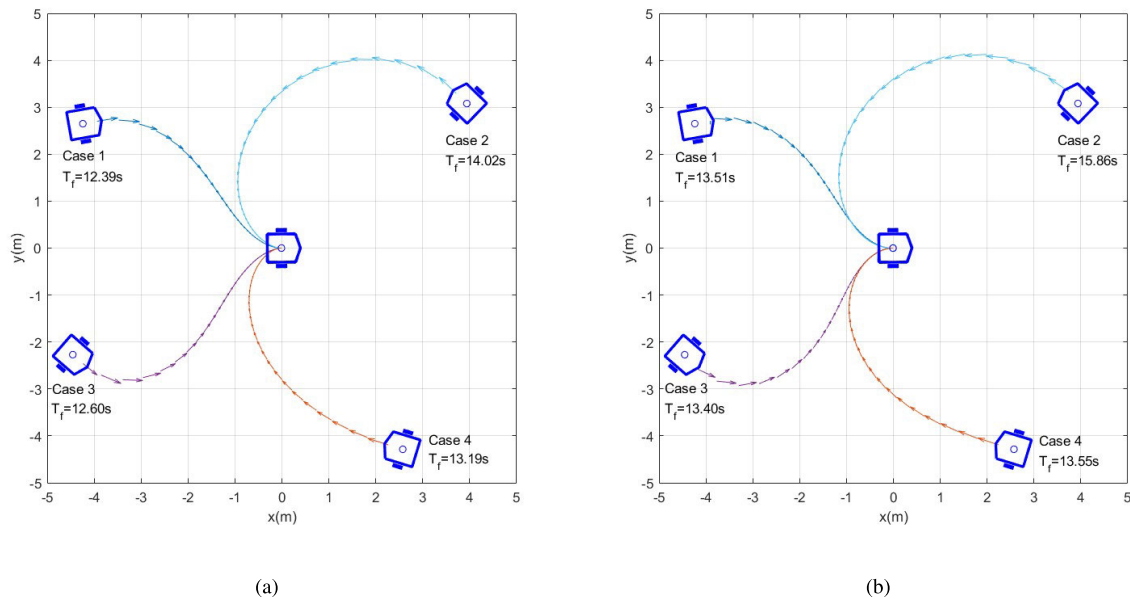


FIGURE 8. Control performance for four initial conditions cases. (a) The proposed method (b) Park's method [4]. The initial posture of the mobile robot is shown in Table 2.

computationally simple and easy to apply, using the optimal control gains obtained from the genetic algorithm across the discretized grid coordinates $N_\alpha \times N_\beta = 72$. This approach allows us to interpolate the optimal control gains for arbitrary initial values. Cubic polynomial interpolation is a method that extracts smooth and continuous polynomials between data points for interpolation. The optimal gains obtained from the genetic algorithm do not differ significantly from surrounding data, making them suitable for application through interpolation algorithms. Fig. 7 illustrates the implementation of the optimal control gains obtained from the 72 two-dimensional space grid points using a cubic polynomial interpolation algorithm. For controlling the mobile robot, the kinematic linear controller gains are determined only once when the robot's initial posture is provided. Despite this, the execution time for polynomial interpolation to calculate these gains averages 25 ms, whereas the neural network method proposed

by Park et al. [4] takes 69 ms. This result indicates that the proposed method is 2.76 times faster. Additionally, the neural network method requires the design and training of the network to obtain the gains, adding to its complexity.

C. PERFORMANCE EVALUATION OF ADAPTIVE DYNAMIC CONTROLLER

When the initial posture of the mobile robot is given, the gains of the kinematic controller are selected using cubic polynomial interpolation, and the kinematic trajectory of the mobile robot is generated. Based on this, the target angular velocity of the mobile robot is calculated, and the mobile robot reaches the target point through the dynamic controller. The adaptive dynamic controller of the mobile robot is an adaptive PID controller based on the gradient descent method, which controls the angular velocities of both wheels of the mobile robot according to the control commands of

TABLE 2. Comparison of optimal control gains and performance of the proposed method with Park's method for the same initial conditions.

Case	Initial Position		Method	Optimal control gains, traveling time						
	α_0	β_0		K_ρ	K_α	K_β	K_P	K_I	K_D	$T_f(\text{sec.})$
1	-42°	32°	The proposed method	0.5177	1.4827	-0.5511	Vary by gradient descent method.			12.39
			Park's method [4]	0.5087	1.3088	-0.5050	0.1668	23.2160	6.7362	13.51
2	25°	145°	The proposed method	0.5175	1.1373	-0.2432	Vary by gradient descent method.			14.02
			Park's method [4]	0.5022	1.1296	-0.3074	5.5129	20.6925	1.2298	15.86
3	55°	-35°	The proposed method	0.5179	1.6267	-0.5748	Vary by gradient descent method.			12.60
			Park's method [4]	0.5100	1.2965	-0.4747	1.0020	16.4273	3.8487	13.40
4	-5°	-125°	The proposed method	0.5177	1.0889	-0.2334	Vary by gradient descent method.			13.19
			Park's method [4]	0.5107	1.4489	-0.4963	4.5056	21.3369	1.8593	13.55

the kinematic controller. The dynamic controller is set with constant gains $[K_P, K_I, K_D]^T = [5, 10, 0.5]^T$ as shown in Equation (10) to ensure a stable start-up in the initial control phase. And, to prevent excessive change in the PID controller gain, the ζ_i of Equation (12) was set small to $[0.0008, 0.0008, 0.0008]^T$. The execution time of the adaptive PID dynamic controller is an average of 0.26 ms, operating within the 10 ms sampling time for mobile robot control, and does not affect the control loop. This result is not significantly different from the average execution time of 0.23 ms for a fixed-gain PID controller.

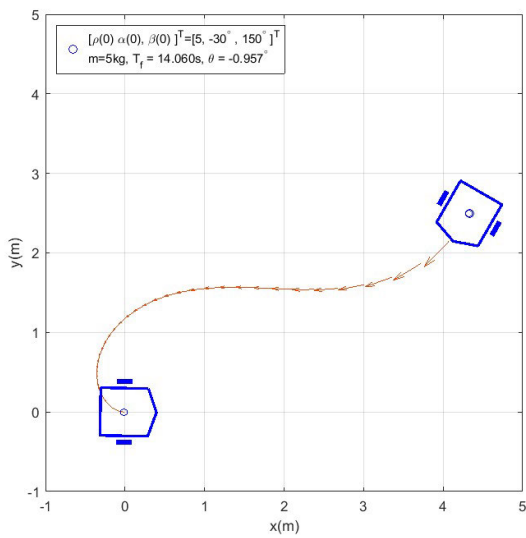
We aim to evaluate the performance of the proposed algorithm for the initial postures of the mobile robot that were not optimized by the genetic algorithm. The proposed algorithm uses a cubic polynomial interpolation technique to obtain the kinematic controller gains, and the dynamic controller applies an adaptive PID controller to evaluate performance across the global positions of the mobile robot. We randomly set the initial posture of the mobile robot in each quadrant and performed a performance evaluation. Fig. 8 compares the results for four arbitrary initial postures with the recent study by Park et al. [4], as shown in Table 2. The study optimized the parameters $([K_\rho, K_\alpha, K_\beta, K_P, K_I, K_D]^T)$ of the kinematic and dynamic controllers using a genetic algorithm for 72 discretized mobile robot postures, and used a neural network for deriving controller gains for arbitrary positions. The experimental results showed that the gains of the kinematic controller were similar in both control methods. Still, the final arrival time indicated better performance in the proposed method than in Park's. These results suggest that the neural network was overfitted or not trained accurately and thus failed to interpolate the optimal gains for an arbitrary initial posture of the mobile robot.

Generally, PID control methods with fixed gains struggle to achieve appropriate gains and performance when dynamic characteristics change, such as the weight of the mobile robot, or when sudden disturbances occur, making it difficult to expect high-performance control. The study by Park's method [4] set the weight of the mobile robot to a constant 20 kg to derive optimal gains through simulation. In practice, the weight of the mobile robot can vary depending on the situation, and dynamic controllers with fixed gains may not yield optimal results under these variations. In contrast, the proposed controller can adaptively change the gains of the

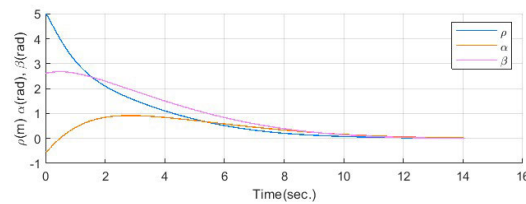
dynamic controller in response to dynamic system variations, such as changes in the weight of the mobile robot, to maintain optimal performance.

Fig. 9 shows the proposed controller's performance evaluation results when the mobile robot's weight is set to 5 kg. The initial posture of the mobile robot was selected as $[\rho(0), \alpha(0), \beta(0)]^T = [5, -30^\circ, 150^\circ]^T$. In the proposed method, the kinematic controller gains $([K_\rho, K_\alpha, K_\beta]^T)$ for trajectory generation were selected as $[(0.5092, 1.2692, -0.03868)]^T$ through cubic polynomial approximation. Fig. 9(a) illustrates the mobile robot's path to the target coordinates, demonstrating that the proposed control method achieved a very stable arrival time of 14.06 s to the target coordinates. Fig. 9(b) shows the trajectory $([\rho(t), \alpha(t), \beta(t)]^T)$ of the mobile robot, indicating very stable characteristics even with dynamic parameter variations. Fig. 9(c) displays the speed and angular velocity of the mobile robot. The proposed controller shows that it stably converges to the target posture within the range of maximum speed and angular velocity constraints, which are dynamic constraints of the mobile robot, especially at the beginning of control. Fig. 9(d) shows the real-time gains of the adaptive PID controller, a dynamic controller, only from 0 to 1.0 s. We can see that the adaptive controller generates appropriate gains for the mobile robot system within 300 ms from the angular velocity error function. Therefore, even when the mobile robot is designed to be very lightweight, the proposed adaptive PID controller can adjust the gains to demonstrate stable driving performance.

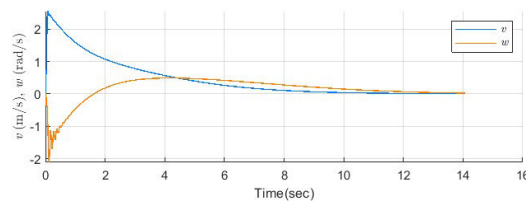
Fig. 10 shows the experimental results from the recent study by Park's method [4], which used a neural network when the mass of the mobile robot was set to a lighter 5 kg, similar to the previous experiment. The controller gains $([K_\rho, K_\alpha, K_\beta, K_P, K_I, K_D]^T)$ determined by the neural network are $[0.5034, 1.1709, -0.03443, 3.5910, 10.2279, 0.9179]$. Fig. 10(a) illustrates the mobile robot's path to the target coordinates. We can see that the mobile robot fails to maintain the exact target posture, deviating by 29.69° even after 30 s. Fig. 10(b) shows the trajectory of the mobile robot at this time. While the mobile robot accurately followed the trajectory for up to 8 s, its trajectory angles $([\alpha(t), \beta(t)]^T)$ increased afterward, failing to maintain a stable target posture. The speed and angle of the mobile robot at this time are shown in Fig. 10(c). Fig. 10(d)



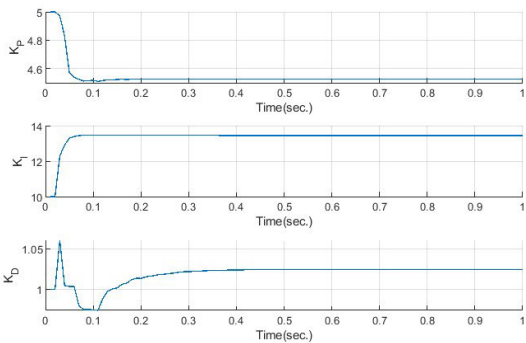
(a)



(b)



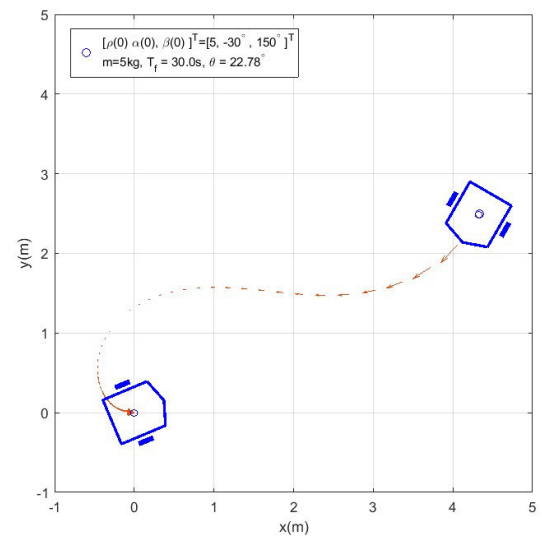
(c)



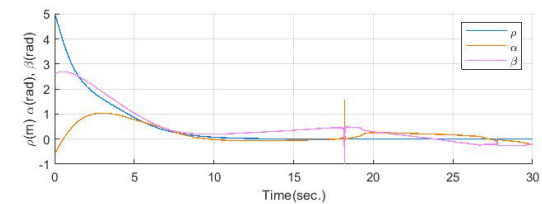
(d)

FIGURE 9. Control performance for a mobile robot with 5kg mass using the proposed method. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Adaptive PID controller gains.

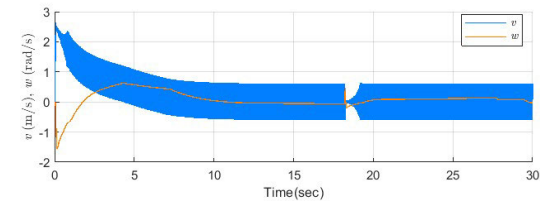
displays the angle $\theta(t)$ concerning the target posture of the mobile robot. These results demonstrate the difficulty for PID



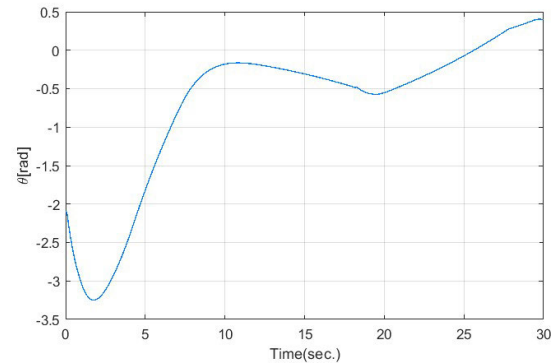
(a)



(b)



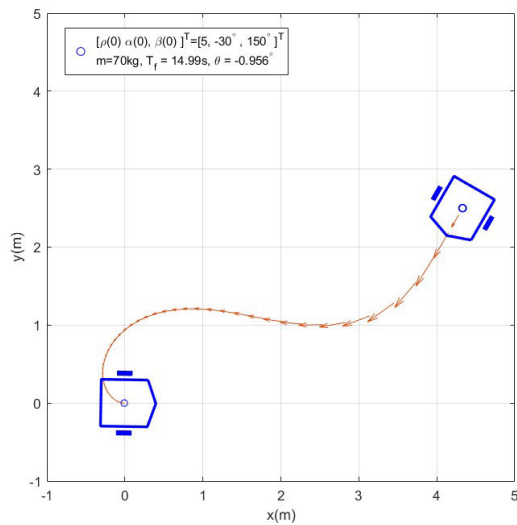
(c)



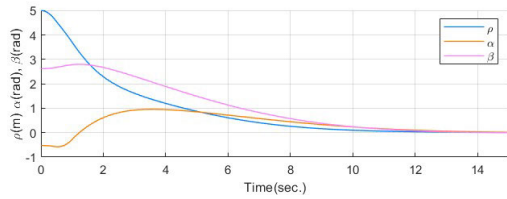
(d)

FIGURE 10. Control performance for a mobile robot with 5kg mass using Park's method [4]. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Angle to target pose.

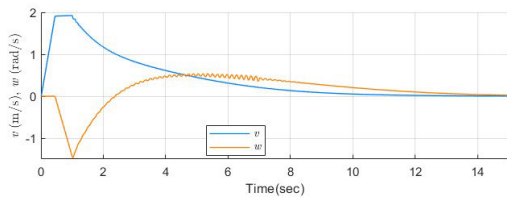
controllers with fixed gains in appropriately responding to dynamic parameter variations. Comparing the gains of each controller, the kinematic controller gains ($[K_\rho, K_\alpha, K_\beta]^T$) of



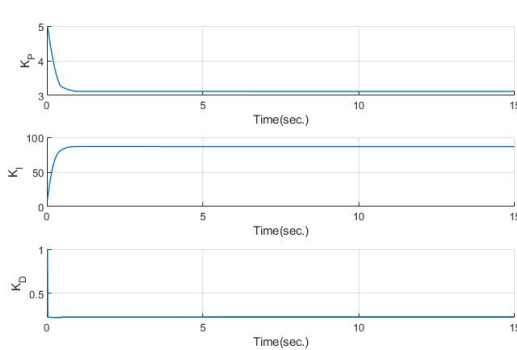
(a)



(b)



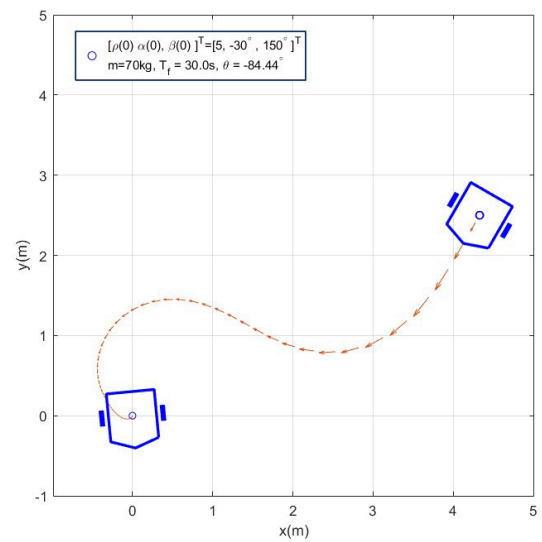
(c)



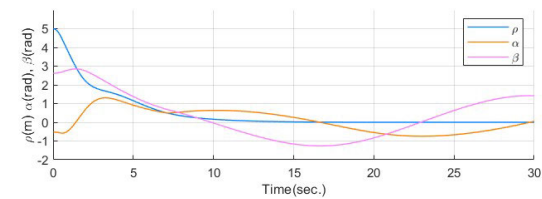
(d)

FIGURE 11. Control performance using the proposed method when a mobile robot with a mass of 70kg and disturbance is applied. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Adaptive PID controller gains.

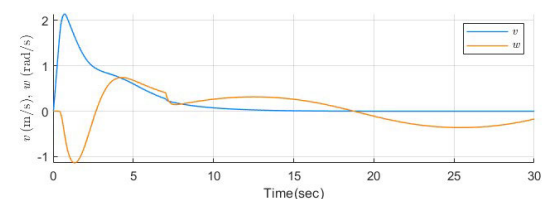
the proposed method is very similar to the gains of the neural network controller of Park's method, but the dynamic controller gains ($[K_P, K_I, K_D]^T$) are very different. There is



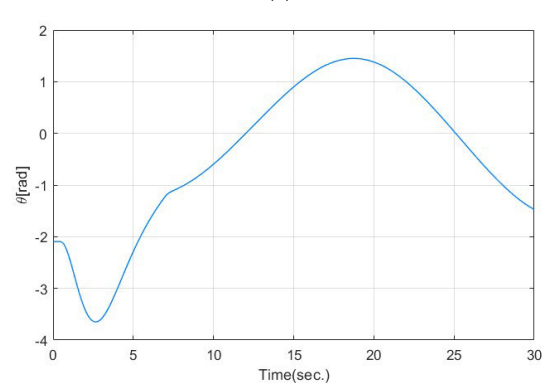
(a)



(b)



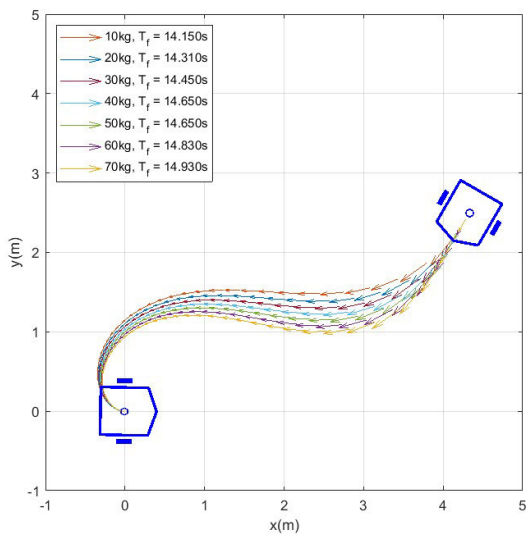
(c)



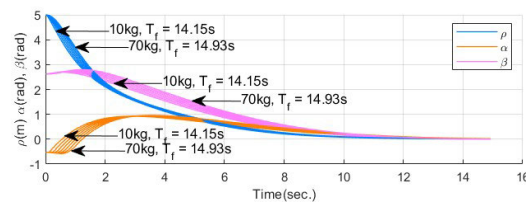
(d)

FIGURE 12. Control performance using Park's method [4] when a mobile robot with a mass of 70kg and disturbance is applied. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Angle to target pose.

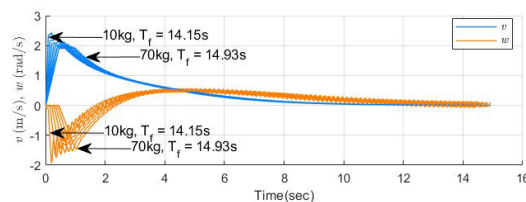
a considerable difference in the proportional and integral gains. Park's method is optimized for the controller design



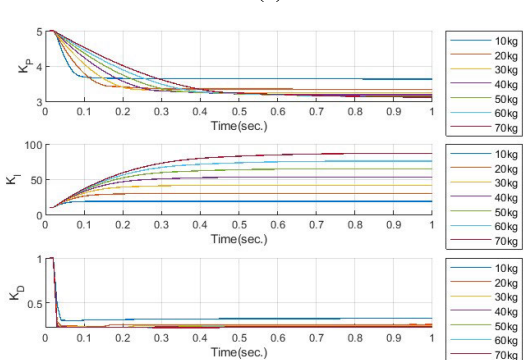
(a)



(b)

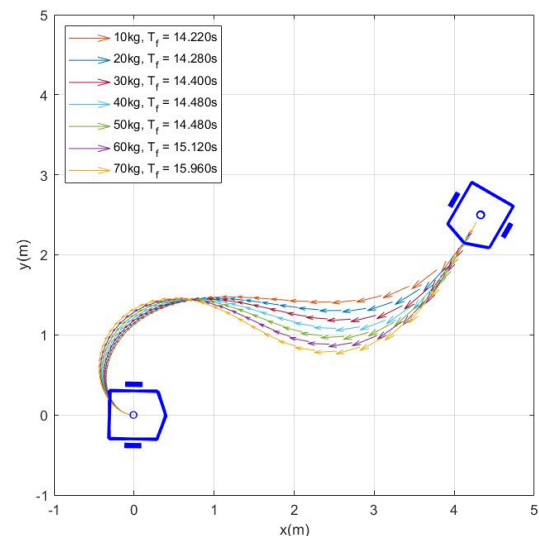


(c)

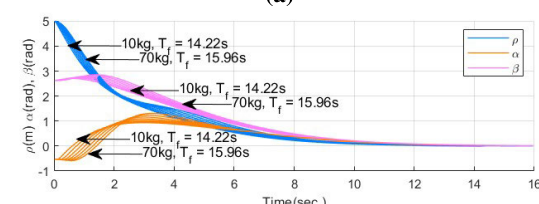


(d)

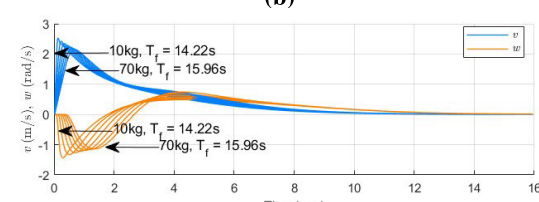
FIGURE 13. Control performance evaluation of the proposed method on mobile robots with various masses. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Adaptive PID controller gains.



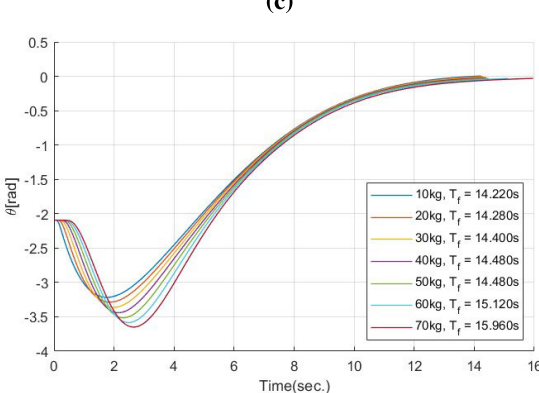
(a)



(b)



(c)



(d)

FIGURE 14. Control performance evaluation of Park's method [4] on mobile robots with various masses. (a) Path of the mobile robot (b) Posture of the mobile robot (c) Velocity and angular velocity (d) Angle to target pose.

of mobile robots with a constant mass, making it challenging to adaptively respond to dynamic parameter variations such

as changes in the robot's mass. Therefore, dynamic controller gains are as crucial as kinematic controller gains for precise control.

Disturbances are external factors that affect the movement and rotation of the mobile robot, typically represented as a vector corresponding to three degrees of freedom, including movement along the x and y axes on a plane and the mobile robot's rotation θ . Disturbances include environmental factors such as wind and ground irregularities, forces from interactions when the robot collides with or pushes objects, and errors or uncertainties in the robot's sensors or actuators [24], [26]. In this study, assuming the control of a medium-sized robot in an indoor environment, a constant disturbance $\tau_d = [0.005N \ 0.005N \ 0.001Nm]^T$ was set for the simulation.

In Figures 11 and 12, the initial pose $([\rho(0), \alpha(0), \beta(0)]^T)$ of the mobile robot is set to $[5, -30, 150]^T$, as in the previous experiment, with the mass of the mobile robot set to 70 kg. Furthermore, starting from the initial pose, a constant disturbance is applied from 7 s onward to compare the performance of the proposed method with Park's method. For trajectory generation of the mobile robot, the kinematic controller gains in the proposed method are selected as $[0.5092 \ 1.2692 \ -0.03868]^T$ through cubic polynomial approximation, while in Park's method, they are selected as $[0.5034 \ 1.1709 \ -0.3443]^T$ using a neural network. Figures 11(a) and 12(a) show the trajectory and speed of the mobile robot using the two methods, indicating the final arrival time and arrival angle. Fig. 11(a) illustrates the path and speed of the mobile robot applying the proposed method, showing it reaches the target coordinates very stable at 14.99 s. In contrast, Fig. 12(a), which applies Park's method, demonstrates that the target pose is not maintained accurately even after 30 s. In trajectory control of the mobile robot, we can see that the proposed method is highly effective and stable compared to Park's method when faced with increases in the mobile robot's mass and disturbances.

Figures 11(b)–(c) and Figures 12(b)–(c) display the pose trajectory $([\rho(t), \alpha(t), \beta(t)]^T)$, velocity $(v(t))$, and angular velocity $(\omega(t))$ of the mobile robot for the two control methods. In Fig. 11(b), the proposed method shows very stable results despite the increased mass of the mobile robot and the presence of disturbances. However, in Fig. 12(b), Park's dynamic controller, which uses optimized constant gains for a mobile robot mass of 20 kg, fails to achieve stable convergence of $[\alpha(t), \beta(t)]^T$ after 7 s. This reason indicates that it does not function effectively when the mass of the mobile robot increases and in the presence of arbitrary disturbances. Figures 11(c) and 12(c) show the velocity and angular velocity of the mobile robot for the two control methods. In Fig. 12(c), a sudden change in angular velocity is observed after the disturbance is applied at 7 s, and it fails to converge even by the final time.

Table 3 shows the results of applying different disturbances to the axes and rotation of the mobile robot. As in the previous experiment, we set the robot's mass to 70 kg and introduced disturbances from the start. Despite these disturbances, the proposed method successfully reached the target coordinates,

with arrival times differing by no more than 0.15 s. The robot's arrival angle was also highly accurate, within about 1 degree of the target posture. In contrast, Park's method failed to reach the final target in two cases where rotational disturbances were applied. In the other four cases where the target was reached, Park's method arrived nearly 1.0 s later than the proposed method.

Figures 13 and 14 set the same initial pose of the mobile robot as in the previous experiments and compare the performance of the proposed method and Park's method as the mass of the mobile robot varies from 10 kg to 70 kg in increments of 10 kg.

Figures 13(a) and 14(a) depict the trajectory and velocity corresponding to changes in the mobile robot's mass, along with the final arrival time. Park's method is faster by several milliseconds compared to the proposed method around the optimized mass of 20 kg. However, when the mass of the mobile robot is either lighter or significantly increased, the trajectory path becomes longer, and the arrival time increases. The proposed method maintains a similar trajectory path despite changes in mass, with a maximum difference in arrival time of 0.78 s. In contrast, Park's method shows a distinct difference in trajectory path and a maximum difference in arrival time of 1.74 s. This difference indicates that the proposed method effectively adapts to changes in the mobile robot's weight. Notably, the maximum arrival time differs by more than twice the amount, with differences of 0.78 s and 1.74 s. Additionally, the proposed method shows very similar trajectories and control states, with the maximum arrival time of 14.93 s being only 1.055 times the minimum arrival time of 14.15 s. In contrast, Park's method exhibits significant differences in trajectory and time, with the maximum arrival time of 15.98 s being 1.124 times the minimum arrival time of 14.15 s. Overall, the proposed method demonstrates greater effectiveness and stability in controlling the mobile robot's trajectory than Park's.

Figures 13(b)–(c) and Figures 14(b)–(c) show the trajectory of the mobile robot's posture $([\rho(t), \alpha(t), \beta(t)]^T)$, velocity $(v(t))$, and angular velocity $(\omega(t))$ for the two control methods. In Fig. 13(b), the proposed method maintains very similar magnitudes and patterns for $[\rho(t), \alpha(t), \beta(t)]^T$, akin to Fig. 9(b), despite variations in the mass of the mobile robot. In contrast, Fig. 14(b) reveals differences according to the weight of the mobile robot when compared to the proposed method. It is evident that Park's dynamic controller, which uses fixed gains, does not operate effectively when the dynamic characteristics of the mobile robot change. Figures 13(c) and 14(c) depict the speed and angular velocity of the mobile robot for the two control methods. Despite a little chattering in the angular velocity changes, the proposed method exhibits high responsiveness in speed and angular velocity. This result is due to the adaptive PID controller's rapid response characteristics, highlighting the proposed method's superior adaptability in handling dynamic changes.

TABLE 3. Comparison of controller performance for various disturbances.

Disturbance: $\tau_d = [\tau_x \quad \tau_y \quad \tau_\theta]^T$	The Proposed method			Park's method [4]		
	Final arrival time: T_f	Final arrival angle: θ_f	Final arrival status	Final arrival time: T_f	Final arrival angle: θ_f	Final arrival status
$[0.0N \quad 0.005N \quad 0.001Nm]^T$	14.94 s	-1.00 °	Success	15.78 s	-1.81 °	Success
$[0.005N \quad 0.0N \quad 0.001Nm]^T$	14.95 s	-0.29 °	Success	15.90 s	0.86 °	Success
$[0.005N \quad 0.005N \quad 0.0Nm]^T$	15.08 s	-0.18 °	Success	30.00 s	15.58 °	Failure
$[0.0N \quad 0.008N \quad 0.002Nm]^T$	14.95 s	-1.00 °	Success	15.98 s	-1.60 °	Success
$[0.008N \quad 0.0N \quad 0.002Nm]^T$	15.07 s	0.02 °	Success	16.30 s	-0.22 °	Success
$[0.008N \quad 0.008N \quad 0.0Nm]^T$	15.09 s	-0.02 °	Success	30.00 s	31.56 °	Failure

Fig. 13(d) illustrates the gain changes of the adaptive PID controller in the proposed method. It shows that as the mass of the mobile robot changes, the gains adaptively change to follow the trajectory of the mobile robot effectively. Fig. 14(d) displays the angles relative to the target pose in Park's method.

As a result of the experiment, the proposed controller is expected to operate efficiently and stably under complex environmental conditions by adjusting controller gains in real time, even in the face of system parameter variations such as the mass of the mobile robot and disturbances. Additionally, the designed controller optimizes only the kinematic controller for trajectory generation using a genetic algorithm, significantly reducing spatial constraints and time compared to existing methods that implement both kinematic and dynamic controllers using a genetic algorithm. Moreover, a well-designed adaptive controller dramatically enhances the system's flexibility and responsiveness, even when the mobile robot operates in various environments or dynamic characteristics such as the robot's mass change. In future research, we aim to extend the optimized kinematic linear controller gains, applied at a fixed position in this study, to optimized linear controllers applicable at arbitrary positions. Through this, we intend to develop a more generalized control method.

VI. CONCLUSION

Ensuring a mobile robot can reach the target position and orientation in the minimum time requires solving a kinematic optimal control problem related to the robot's initial pose. The optimal control gains for a mobile robot are analytically challenging to determine, and kinematic analysis, especially when considering dynamic models, is even more complex. This study aims to design a controller from two perspectives. First, it seeks to determine the optimal control gains for a kinematic linear controller using a genetic algorithm and to derive efficient control gains under various initial postures by employing cubic polynomial interpolation, enabling mobile robots to achieve the best kinematic gains from any position. Second, it aims to design a stable and robust dynamic controller using an adaptive PID controller despite changes in system parameters such as the mass of the mobile robot and disturbances.

Simulation results demonstrate that the proposed kinematic controller generates optimal trajectories more

effectively than existing methods, and the adaptive PID dynamic controller maintains optimal performance despite disturbances and changes in the robot's mass. In experiments with various disturbances, the proposed method consistently reached target coordinates, with arrival times differing by no more than 0.15 s. In contrast, the comparative method failed to reach the target in two cases with rotational disturbances and was nearly 1.0 s slower in the other four cases. Additionally, when varying robot masses, the proposed and comparative methods showed a difference of over twice in maximum arrival times, with differences of 0.78 s and 1.74 s, respectively. The proposed method maintained very similar trajectories and control states, with a maximum arrival time of 14.93 s and only 1.055 times the minimum arrival time of 14.15 s.

Furthermore, the kinematic controller designed using a genetic algorithm significantly reduces spatial constraints and learning time compared to existing methods that include dynamic controllers. These findings contribute significantly to the practical design of mobile robot controllers, ensuring efficient and stable operation, particularly in scenarios requiring rapid derivation of optimal gains. Future work should focus on designing an intelligent controller that integrates kinematic and dynamic controllers for mobile robots with arbitrary positions and postures. Meta-learning, in particular, enables both online and offline learning and could offer a valuable approach for integrating these two controllers.

REFERENCES

- [1] Z. Sun, S. Hu, H. Xie, H. Li, J. Zheng, and B. Chen, "Fuzzy adaptive recursive terminal sliding mode control for an agricultural omnidirectional mobile robot," *Comput. Electr. Eng.*, vol. 105, Jan. 2023, Art. no. 108529, doi: [10.1016/j.compeleceng.2022.108529](https://doi.org/10.1016/j.compeleceng.2022.108529).
- [2] K. Lee and S. Park, "Geometric zone-control algorithm for collision and deadlock avoidance in AGV system," *IEEE Access*, vol. 11, pp. 131289–131301, 2023, doi: [10.1109/ACCESS.2023.3335843](https://doi.org/10.1109/ACCESS.2023.3335843).
- [3] I. A. Sulistijono, M. Rois, A. Yuniawan, and E. H. Binugroho, "Teleoperated food and medicine courier mobile robot for infected diseases patient," in *Proc. IEEE Region 10 Conf. (TENCON)*, Dec. 2021, pp. 620–625, doi: [10.1109/TENCON54134.2021.9707377](https://doi.org/10.1109/TENCON54134.2021.9707377).
- [4] S.-C. Park, M. Kim, H.-M. Park, and J.-H. Park, "Optimal control gains optimization for mobile robot considering dynamic constraints," *IEEE Access*, vol. 12, pp. 180079–180092, 2024, doi: [10.1109/ACCESS.2024.3507739](https://doi.org/10.1109/ACCESS.2024.3507739).
- [5] S. J. Al-Kamil and R. Szabolcsi, "Optimizing path planning in mobile robot systems using motion capture technology," *Results Eng.*, vol. 22, Jun. 2024, Art. no. 102043, doi: [10.1016/j.rineng.2024.102043](https://doi.org/10.1016/j.rineng.2024.102043).

- [6] J. Piaskonka, "Different kinematic path following controllers for a wheeled mobile robot of (2,0) type," *J. Intell. Robotic Syst.*, vol. 77, nos. 3–4, pp. 481–498, Mar. 2015, doi: [10.1007/s10846-013-9879-6](https://doi.org/10.1007/s10846-013-9879-6).
- [7] D. Wang, J. Zhou, C. He, and Z. Miao, "Experimental verification of formation control in nonholonomic multi-mobile robots," in *Proc. 40th Chin. Control Conf. (CCC)*, Jul. 2021, pp. 5327–5331, doi: [10.23919/CCCS2363.2021.9550119](https://doi.org/10.23919/CCCS2363.2021.9550119).
- [8] I. Negrean, C. Schonstein, K. Kacso, C. Negrean, and A. Duca, "Formulations about dynamics of mobile robots," *Solid State Phenomena*, vols. 166–167, pp. 309–314, Sep. 2010, doi: [10.4028/www.scientific.net/ssp.166-167.309](https://doi.org/10.4028/www.scientific.net/ssp.166-167.309).
- [9] Y. Choi and J. Park, "Predictive control based on genetic algorithm for mobile robots with constraints," *J. Korea Inst. Inf. Commun. Eng.*, vol. 22, no. 1, pp. 9–16, Jan. 2018, doi: [10.6109/jkiice.2018.22.1.9](https://doi.org/10.6109/jkiice.2018.22.1.9).
- [10] R. Fierro and F. L. Lewis, "Control of a nonholonomic mobile robot using neural networks," *IEEE Trans. Neural Netw.*, vol. 9, no. 4, pp. 589–600, Jul. 1998, doi: [10.1109/72.701173](https://doi.org/10.1109/72.701173).
- [11] S. Nandy, S. N. Shome, G. Chakraborty, and C. S. Kumar, "A modular approach to detailed dynamic formulation and control of wheeled mobile robot," in *Proc. IEEE Int. Conf. Mechatronics Autom.*, Aug. 2011, pp. 1471–1478, doi: [10.1109/ICMA.2011.5985967](https://doi.org/10.1109/ICMA.2011.5985967).
- [12] F. N. Martins and A. S. Brandão, "Motion control and velocity-based dynamic compensation for mobile robots," in *Applications of Mobile Robots*. London, U.K.: IntechOpen, 2019.
- [13] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion planning and control for mobile robot navigation using machine learning: A survey," *Auto. Robots*, vol. 46, no. 5, pp. 569–597, Mar. 2022.
- [14] Y. Cao, K. Ni, T. Kawaguchi, and S. Hashimoto, "Path following for autonomous mobile robots with deep reinforcement learning," *Sensors*, vol. 24, no. 2, p. 561, Jan. 2024, doi: [10.3390/s24020561](https://doi.org/10.3390/s24020561).
- [15] R. Singh, T. K. Bera, and N. Chatti, "A real-time obstacle avoidance and path tracking strategy for a mobile robot using machine-learning and vision-based approach," *Simulation*, vol. 98, no. 9, pp. 789–805, Apr. 2022, doi: [10.1177/00375497221091592](https://doi.org/10.1177/00375497221091592).
- [16] R. Chai, H. Niu, J. Carrasco, F. Arvin, H. Yin, and B. Lennox, "Design and experimental validation of deep reinforcement learning-based fast trajectory planning and control for mobile robot in unknown environment," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5778–5792, Apr. 2022.
- [17] H. Liu, Y. Shen, S. Yu, Z. Gao, and T. Wu, "Deep reinforcement learning for mobile robot path planning," 2024, *arXiv:2404.06974*.
- [18] É. Poulin, A. Pomerleau, A. Desbiens, and D. Hodouin, "Development and evaluation of an auto-tuning and adaptive PID controller," *Automatica*, vol. 32, no. 1, pp. 71–82, Jan. 1996.
- [19] H. Chang and T.-S. Jin, "Adaptive tracking controller based on the PID for mobile robot path tracking," in *Proc. 6th Int. Conf. Intell. Robot. Appl. (ICIRA)*, Busan, South Korea. Berlin, Germany: Springer, Jan. 2013, pp. 540–549.
- [20] A. S. Al-Araji, M. F. Abbod, and H. S. Al-Raweshidy, "Design of an adaptive nonlinear PID controller for nonholonomic mobile robot based on posture identifier," in *Proc. IEEE Int. Conf. Control Syst., Comput. Eng.*, Nov. 2011, pp. 337–342.
- [21] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Trans. Control Syst. Technol.*, vol. 15, no. 3, pp. 566–580, May 2007, doi: [10.1109/TCST.2007.894653](https://doi.org/10.1109/TCST.2007.894653).
- [22] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed., Cambridge, MA, USA: MIT Press, 2011.
- [23] F. L. Lewis, D. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed., Hoboken, NJ, USA: Wiley, 2012.
- [24] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*, vol. 3. New York, NY, USA: Wiley, 2006, pp. 75–118.
- [25] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. Cambridge, MA, USA: MIT Press, 2005.
- [26] G. Cook and F. Zhang, *Mobile Robots: Navigation, Control and Sensing, Surface Robots and AUVs*. Hoboken, NJ, USA: Wiley, 2020.
- [27] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA, USA: MIT Press, 1998.
- [28] J. Holland, "Genetic algorithms," *Sci. Amer.*, vol. 267, no. 1, pp. 44–50, 1992.
- [29] (2024). *MathWorks*. Accessed: 2025. [Online]. Available: <https://kr.mathworks.com/>



JIN-HYUN PARK received the B.S., M.S., and Ph.D. degrees in electrical engineering from Pusan National University, Pusan, South Korea, in 1992, 1994, and 1997, respectively. He is currently a Professor with the School of Mechatronics Engineering, Gyeongsang National University. His research interests include robotics, intelligent control, and deep learning.

...